Uppsala University



Machine Learning

Assignment 1

General information

- The recommended tool in this course is R (with the IDE R-Studio). You can download R **here** and R-Studio **here**. You can use Python and Jupyter Notebooks, although the assignments may use data available only through the R package, a problem you would need to solve yourself.
- Report all results in a single, `*.pdf`-file. *Other formats, such as Word, Rmd, Jupyter Notebook, or similar, will automatically be failed.* Although, you are allowed first to knit your document to Word or HTML and then print the assignment as a PDF from Word or HTML if you find it difficult to get TeX to work.
- The report should be written in English.
- Print the complete definition of every function you implement in the PDF report, including functions in your chosen VG tasks and any helper functions you write. Show each definition once, next to the relevant answer or in a clearly referenced appendix. Function calls, numerical output, and passing automatic checks do not replace the function definitions. In R Markdown, keep these definitions visible with `echo = TRUE`; do not hide them with `echo = FALSE` or `include = FALSE`.
- If a question is unclear in the assignment. Write down how you interpret the question and formulate your answer.
- You should submit the report to **Studium**. Deadlines for all assignments are **Sunday 23.59**. See **Studium** for dates. Assignments will be graded within 10 working days from the assignment deadline.
- To pass the assignments, *you should answer all questions not marked with **, and get at least 75% correct.
- To get VG on the assignment, *also the questions marked with a ** should be answered. Sometimes you can choose between different VG assignments, then this is explicitly stated. To get VG you need 75% both on questions to pass and the VG part of the assignment. VG will only be awarded on the first deadline of the assignment.
- A report that does not contain the general information (see the **template**), will be automatically rejected.
- Every assignment report must include one oral exam question of your own together with a short model answer, following the oral exam question contribution instructions in this assignment. You are not allowed to use large language models, such as ChatGPT, at all for this contribution.
- State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.
- When working with R, we recommend writing the reports using R markdown and the provided **R markdown template**. The template includes the formatting instructions and how to include code and figures.

- Instead of R markdown, you can use other software to make the pdf report, but you should use the same instructions for formatting. These instructions are also available in **the PDF produced from the R markdown template**.
- The course has its own R package `uuml` with data and functionality to simplify coding. To install the package just run the following:

  1. `install.packages("remotes")`
  2. `remotes::install_github("MansMeg/IntroML", subdir = "rpackage")`

- We collect common questions regarding installation and technical problems in a course Frequently Asked Questions (FAQ). This can be found **here**.
- You are not allowed to show your assignments (text or code) to anyone. Only discuss the assignments with your fellow students. The student that show their assignment to anyone else could also be considered to cheat. During scheduled support sessions held on Zoom, only share your screen in a separate room with a teaching assistant.
- You may use large language models (LLMs), such as ChatGPT, like you may discuss with fellow students: to learn, ask for explanations, and get feedback. You may not use LLMs to produce the solution you submit. You must be able to explain every part of your own assignment at the oral exam, including your code, results, and reasoning, even when it is not covered by the published oral exam questions. If course staff have concerns about your understanding or the authorship of any part of your submission, you may be asked to explain that part at the oral exam.
- Where an exercise explicitly asks you to run an LLM and report its output, you may do so for that experiment. Your explanations and analysis must be your own. This permission does not apply to the oral exam question contribution, for which LLM use is prohibited entirely.
- Do not copy or upload the full assignment text into an LLM.
- Scheduled support sessions are for questions about the assignments and course material. Do not hesitate to ask! Ask questions early if instructions are unclear or you encounter possible bugs, so you do not lose time unnecessarily.
- If you have any suggestions or improvements to the course material, please post in the course chat feedback channel, create an issue **here**, or submit a pull request to the public repository.

Contents

Automatic feedback on your R functions **3**

Oral exam question contribution **4**

1 Basic, Stochastic, and Mini-Batch Gradient Descent **5**

1.1 Implement the gradient for logistic regression 5

1.2 Implement Gradient Descent 6

2 Regularized Regression **10**

3 * Gradient Descent for penalized logistic regression **12**

4 * Implementation of the Adam optimizer **15**

Automatic feedback on your R functions

The R package `markmyassignment` (MMA) provides automatic feedback on the functions listed in the selected check configuration. Define or load your own functions in R before running the checks. Use the `tasks` argument to check one function by name; omitting it checks every function in the selected configuration. Functions that depend on other functions need those definitions loaded too.

To select the checks for this assignment, run the following code in R:

```
install.packages("markmyassignment") # Install once
library(markmyassignment)
assignment_path <-
  paste("https://github.com/MansMeg/IntroML/",
        "blob/master/assignments/tests/assignment1/assignment1.yml", sep="")
set_assignment(assignment_path)
# List the function names available in this configuration
show_tasks()
# To check one function, replace function_name with its name:
# mark_my_assignment(tasks = "function_name")
# To check your code/functions, just run
mark_my_assignment()
```

The checks provide feedback on selected numerical cases; they do not assess the whole assignment or award marks. Print your complete function definitions in the PDF report as required by the general instructions, even when the checks pass. You do not need to include MMA results in the report. The supplied checks run R code; Python implementations are assessed from the report and cannot be run directly through MMA.

Oral exam question contribution

Write one possible oral exam question of your own for this assignment together with a short model answer. Include both in your assignment report.

The question should:

- concern material from this assignment or the associated reading or lectures,
- be suitable for a 2–4 minute oral answer, possibly using a whiteboard,
- test conceptual understanding rather than only calculation or code,
- not already be included in the oral exam question list and not be very close to an existing question (e.g. just different numbers).

Note! You are not allowed to use large language models, such as ChatGPT, at all for this oral exam question contribution.

State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.

1 Basic, Stochastic, and Mini-Batch Gradient Descent

Implement and compare gradient descent methods for logistic regression. See Ruder (2016) for pseudo-code and details on the algorithms.

Training a neural network follows the same pattern: define a loss, compute its gradients with respect to the weights and biases, and update them using an optimiser such as mini-batch gradient descent or Adam. Logistic regression provides a small model in which you can inspect these steps directly. In later neural-network assignments, libraries use backpropagation to compute the gradients automatically; the learning-rate, batch-size, and regularisation choices explored here remain important.

We are going to work with this data as a test case:

```
library(uuml)
data("binary")

binary$gre_sd <- (binary$gre - mean(binary$gre))/sd(binary$gre)
binary$gpa_sd <- (binary$gpa - mean(binary$gpa))/sd(binary$gpa)
X <- model.matrix(admit ~ gre_sd + gpa_sd, binary)
y <- binary$admit
```

1.1 Implement the gradient for logistic regression

The likelihood function for logistic regression is

$$L(\theta, \mathbf{y}, \mathbf{X}) = \prod_{i=1}^{n} p_i^{y_i} (1 - p_i)^{1 - y_i} \,,$$

where

$$\log \frac{p_i}{1 - p_i} = \mathbf{x}_i \theta \,,$$

and $\mathbf{x}_i$ is the ith row of the design matrix $\mathbf{X}$ and $\theta \in \mathbb{R}^P$ is a $P \times 1$ column vector of parameters.

Maximizing the likelihood is equivalent to minimizing the *mean negative log-likelihood*, which we denote by NLL throughout this assignment:

$$\begin{aligned} \text{NLL}(\theta, \mathbf{y}, \mathbf{X}) &= -\frac{1}{n} \log L(\theta, \mathbf{y}, \mathbf{X}) \\ &= -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right] \\ &= \frac{1}{n} \sum_{i=1}^{n} \left[\log(1 + \exp(\mathbf{x}_i \theta)) - y_i \mathbf{x}_i \theta \right] . \end{aligned} \tag{1}$$

This is the binary cross-entropy averaged over observations. Both the loss and its gradient use this mean convention; no additional division by n is needed after computing either one.

1.1a. (4p) Derive the gradient of the mean negative log-likelihood $\text{NLL}(\theta, \mathbf{y}, \mathbf{X})$ with respect to θ.
Hint! See Hastie et al. (2009, Ch. 4.4).

1.1b. (2p) Implement `nll_grad(y, X, theta)`, returning the mean-loss gradient as a numeric vector in the same order as the columns of `X`. Include the function code and your actual outputs for the two calls below; compare them with the supplied results to four decimal places.

```
nll_grad(y, X, theta = c(0,0,0))

## (Intercept)      gre_sd      gpa_sd
##      0.1825     -0.0857     -0.0829
```

```
nll_grad(y, X, theta = c(-1,0.5,0.5))

## (Intercept)      gre_sd      gpa_sd
##     -0.0217      0.0395      0.0426
```

Check: `mark_my_assignment(tasks = "nll_grad")`

1.2 Implement Gradient Descent

Use the following function to evaluate the mean negative log-likelihood. It converts the total log-likelihood supplied by `uuml` to the mean loss defined above. Use `nll()` for loss evaluation throughout this assignment.

```
nll <- function(y, X, theta){
  -uuml::ll(y, X, theta) / length(y)
}
theta <- c(0, 0, 0)
nll(y, X, theta)

## [1] 0.6931472
```

Hint! Gradient descent subtracts the gradient of the loss. Since `nll_grad()` already returns the gradient of the mean negative log-likelihood, do not negate it or divide it by the number of observations again.

For SGD and mini-batch updates, evaluate `nll_grad()` on the current observation or mini-batch, averaging over the observations supplied to that call. To monitor training, evaluate `nll()` on *all training observations* at the end of each epoch, so every algorithm's loss history uses the same scale.

1.2a. (1p) Fit logistic regression using `glm()` with `family = "binomial"`, and save the model as `glm_fit`. Print the three maximum-likelihood parameter estimates.

Hint! The design matrix `X` already includes an intercept so you need to remove that either from `X` or from the R formula using `y ~ -1 + X`.

1.2b. (6p) Implement *one* R function called `gradient_descent()`. Use the arguments shown in the scaffold below and the gradient of the mean NLL (see Ruder, 2016, for algorithm descriptions). The batch size determines the method:

- `batch_size = nrow(X)` gives ordinary full/batch gradient descent;
- `batch_size = 1` gives stochastic gradient descent;
- `batch_size = 10` gives mini-batch gradient descent.

Print the function once in your report. Reuse the figures and explanations from Experiments A and B as evidence, referring to them where relevant; no extra runs or output tables are required. Run for the specified number of epochs and assess convergence from the trajectories.

Each epoch must process every observation exactly once. For batches smaller than n, shuffle the observation indices once per epoch and split them into non-overlapping batches. Include any smaller final batch when n is not divisible by the batch size. Make one parameter update per batch, using the mean gradient of that batch, and record the full-training-data mean NLL and parameters after the complete epoch.

Example: One epoch and gradient dimensions *No submission required*

Five observations, three coefficients (intercept and two slopes), batch size 2, and shuffled order $4, 1, 5, 2, 3$ give:

Update	Rows	Shape of batch X	Gradient length
1	$4, 1$	2×3	3
2	$5, 2$	2×3	3
3	3	1×3	3

One epoch makes three updates. Average each gradient over its actual batch size; record the full-data mean NLL after update 3.

Hint! Use `X[batch, , drop = FALSE]` when selecting rows so a batch of size one remains a matrix. Start from $\theta = 0$ for every call. Complete the batching and update steps in this scaffold:

```r
gradient_descent <- function(y, X, batch_size, eta, epochs){
  results <- matrix(0.0, ncol = ncol(X) + 2L, nrow = epochs)
  colnames(results) <- c("epoch", "nll", colnames(X))
  theta <- rep(0.0, ncol(X))
  for(epoch in seq_len(epochs)){

    # Choose the observation order and split it into batches.
    # For each batch, use nll_grad() to update theta.

    results[epoch, "epoch"] <- epoch
    results[epoch, "nll"] <- nll(y, X, theta)
    results[epoch, -(1:2)] <- theta
  }
  return(results)
}
```

Check: `mark_my_assignment(tasks = "gradient_descent")`

Once implemented, the same function produces the three histories for Experiment B:

```
full_run <- gradient_descent(y, X, batch_size = nrow(X),
                             eta = 0.1, epochs = 500)
set.seed(4711)
single_run <- gradient_descent(y, X, batch_size = 1,
                               eta = 0.1, epochs = 500)
set.seed(4711)
mini_run <- gradient_descent(y, X, batch_size = 10,
                             eta = 0.1, epochs = 500)
```

1.2c. (4p) **Experiment A: learning rates with full gradient descent.** Set the batch size to n and run the function for 500 epochs from $\theta = 0$, keeping the learning rate fixed within each run. Choose three learning rates that illustrate slow progress, useful convergence, and unstable or oscillating behaviour. Only full GD is needed for this experiment.

Submit *one composite figure*: use three columns for the learning-rate settings and two rows for the mean NLL and one coefficient of your choice. Record the actual learning rates in the panel labels. Add the corresponding estimates from `glm()` as reference lines: its mean NLL in the loss panels and its coefficient estimate in the coefficient panels. Separate vertical scales are allowed so an unstable run does not hide the other curves.

Briefly explain how the three learning rates affect the loss and coefficient trajectories. A persistently oscillating loss is sufficient to demonstrate instability; you do not need to force the loss to become infinite.

1.2d. (4p) **Experiment B: batch sizes at a shared learning rate.** Call the same `gradient_descent()` function with batch sizes n, 10, and 1 to compare full GD, mini-batch GD, and SGD. Use the same data and 500 epochs for all three. Start with a shared learning rate of $\eta = 0.1$. If any run is unstable, reduce the learning rate for *all three* methods. Set the random seed before each stochastic run and report the common learning rate and seed.

Submit *one composite figure with two panels*: overlay the three methods' mean-NLL curves in one panel and their trajectories for the same coefficient used in Experiment A in the other. Use a legend and the `glm()` reference lines.

Compare progress and fluctuations near the reference estimates. Report the number of parameter updates per epoch for each method. An epoch processes all n observations, but full GD makes one update, mini-batch GD makes $\lceil n/10 \rceil$ updates, and SGD makes n updates. Explain why equal epochs do not mean equal numbers of updates; this experiment is not a wall-clock timing comparison.

Plotting hint for the two experiments

Reuse your saved mean-loss and coefficient histories; no additional optimisation runs are needed for plotting. To find the three learning-rate behaviours, try a short logarithmic sequence of learning rates, such as 0.001, 0.01, 0.1, 1, and 10, then adjust only if needed.

Collect the three result matrices for one experiment in a named list called `runs`. For example, its names can describe the learning rates in Experiment A or the methods in Experiment B. Each matrix should have the columns `epoch`, `nll`, and the parameter names from `X`. The following code only reshapes and plots results; use the output of your own optimizers.

```
library(ggplot2)
# Here full_run, mini_run, and single_run are your saved histories.
runs <- list("Full GD" = full_run, "Mini-batch GD" = mini_run,
             "SGD" = single_run)
coefficient <- "gre_sd" # Choose one coefficient for both experiments.
plot_data <- do.call(rbind, lapply(names(runs), function(label){
  h <- runs[[label]]
  rbind(
    data.frame(epoch = h[, "epoch"], value = h[, "nll"],
               measure = "Mean NLL", setting = label),
    data.frame(epoch = h[, "epoch"], value = h[, coefficient],
               measure = "Coefficient", setting = label)
  )
}))
plot_data$measure <- factor(plot_data$measure,
                            levels = c("Mean NLL", "Coefficient"))
plot_data$setting <- factor(plot_data$setting, levels = names(runs))
```

Construct the two reference values from `glm_fit` and plot Experiment B as follows:

```
theta_mle <- unname(coef(glm_fit)) # Same parameter order as X.
reference <- data.frame(
  measure = factor(c("Mean NLL", "Coefficient"),
                   levels = c("Mean NLL", "Coefficient")),
  value = c(nll(y, X, theta_mle),
            theta_mle[match(coefficient, colnames(X))])
)
ggplot(plot_data, aes(epoch, value, colour = setting)) +
  geom_line() +
  geom_hline(data = reference, aes(yintercept = value),
             linetype = "dashed") +
  facet_wrap(~ measure, ncol = 1, scales = "free_y") +
  labs(x = "Epoch", y = NULL, colour = "Setting")
```

For Experiment A, prepare `runs` with its three learning-rate results and replace the facet line with `facet_wrap(~ measure + setting, ncol = 3, scales = "free_y")`. This produces the six panels in one figure. You can choose another plotting tool or layout that makes the same comparisons clear.

2 Regularized Regression

The datasets `prob2_train` and `prob2_test` contain simulated data with 240 explanatory variables (`V1-V240`) and 1 numerical response variable (`y`). As per the dataset names, the first dataset contains training data and the second contains test data for this problem. To access the data, just run:

```
library(uuml)
data("prob2_train")
data("prob2_test")
dim(prob2_train)

## [1] 200 241

X <- as.matrix(prob2_train[,-241])
y <- as.matrix(prob2_train["y"])
X_test <- as.matrix(prob2_test[,-241])
y_test <- as.matrix(prob2_test["y"])
```

Example: Non-unique coefficients *No submission required*

With two observations and three coefficients, consider

$$X = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}, \qquad \theta_A = (1,1,0)^\top, \quad \theta_B = (0,2,1)^\top.$$

Both give $X\theta = (1,2)^\top$. For a new row $(1,0,0)$, they predict 1 and 0, respectively. A perfect training fit can coexist with non-unique coefficients; here $X^\top X$ is singular.

You should do the following and present the results in your report:

2a. (1p) Fit a linear model to the training data. What are the results? Why does this happen?

2b. (1p) Use `glmnet()` function from the `glmnet` package to fit a linear lasso regression to the training data with $\lambda = 1$. How many predictor coefficients are nonzero, excluding the intercept? *Hint!* `coefficients()`

2c. Split the data into ten folds randomly. *Hint!* Use the `sample()` function and create a new variable called fold. Here is an example on how to select the folds

```
fold <- sample(rep(1:10, length.out = nrow(X)))

X_trainfold1 <- X[fold!=1,]
X_valfold1 <- X[fold==1,]
y_trainfold1 <- y[fold!=1,]
y_valfold1 <- y[fold==1,]
```

2d. (5p) Implement 10-fold cross-validation on the training data. Include the function code once and use the $\lambda = 1$ result requested below to demonstrate it. *Hint!* Implement this as `glmnet_cv(fold, X, y, lambda)`. Use the supplied ten folds and return one number: the mean of the ten validation-fold RMSEs. Accept either a response vector or a one-column response matrix. Use lasso (`alpha = 1`) with the default Gaussian family, intercept, and standardisation in `glmnet()`. Keep the same folds when comparing different values of `lambda`.

2d.i. For each fold, hold out the validation fold and train on the rest of the training set. See below.

```
mod <- glmnet(X_trainfold, y_trainfold,
              alpha = 1, lambda = lambda)
pred_y_valfold <- predict(mod, newx = X_valfold)
```

2d.ii. Use the `rmse()` function in the `uuml` R package to compute the root mean squared error (RMSE) for each fold as follows:

```
rmse(pred_y_valfold, y_valfold)
```

2d.iii. Take the mean RMSE for the 10 folds.

Check: `mark_my_assignment(tasks = "glmnet_cv")`

2e. (1p) Now, compute the RMSE for $\lambda = 1$ using 10-fold cross-validation.

2f. (2p) Test different values for the hyper parameter λ. What is the best RMSE you can get with 10-fold cross-validation? What is the λ for this value?

2g. (1p) Refit the model with the selected λ on all training observations, then report its RMSE on the test set. Use the test set only for this final evaluation.

3 * Gradient Descent for penalized logistic regression

For the optional distinction task, complete either this section or Section 4.

Reuse the binary-admission data from Section 1.

```
library(uuml)
data("binary")

binary$gre_sd <- (binary$gre - mean(binary$gre))/sd(binary$gre)
binary$gpa_sd <- (binary$gpa - mean(binary$gpa))/sd(binary$gpa)
X <- model.matrix(admit ~ gre_sd + gpa_sd, binary)
y <- binary$admit
```

As in Section 1, we minimize the mean negative log-likelihood. With a ridge penalty, the loss is

$$\mathrm{NLL}_r(\theta, \mathbf{y}, \mathbf{X}, \lambda) = \mathrm{NLL}(\theta, \mathbf{y}, \mathbf{X}) + \frac{\lambda}{2} \sum_{j=2}^{P} \theta_j^2 , \tag{2}$$

where NLL is the mean negative log-likelihood defined in Section 1 and $\lambda \geq 0$. The first column of $\mathbf{X}$ is the intercept column, so θ_1 is excluded from the penalty (see Hastie et al., 2009, Ch. 3.4).

Example: The unpenalized intercept *No submission required*

For $\theta = (2, 0.4, -0.3)^\top$ (intercept first) and $\lambda = 1$, the ridge penalty is

$$\frac{1}{2}\left(0.4^2 + (-0.3)^2\right) = 0.125.$$

Changing the intercept from 2 to 20 leaves this penalty unchanged, although the likelihood loss can change.

This scaling matches the logistic ridge objective in `glmnet`; changing the likelihood scaling changes the meaning of λ. More information can be found here.

3a. (2p) Derive the gradient for $\mathrm{NLL}_r(\theta, \mathbf{y}, \mathbf{X}, \lambda)$ with respect to θ.

3b. (1p) Implement the gradient as `nllr_grad(y, X, theta, lambda)`. It should return the gradient of NLL_r, including the penalty, with the intercept unpenalized. Include the function code and your actual outputs for the three calls below; compare them with the supplied results to four decimal places.

```
nllr_grad(y, X, theta = c(0,0,0), lambda = 0)

## (Intercept)      gre_sd      gpa_sd
##      0.1825     -0.0857     -0.0829
```

```
nllr_grad(y, X, theta = c(0,0,0), lambda = 1)

## (Intercept)      gre_sd      gpa_sd
##      0.1825     -0.0857     -0.0829
```

```
nllr_grad(y, X, theta = c(-1,0.5,0.5), lambda = 1)

## (Intercept)      gre_sd      gpa_sd
##     -0.0217      0.5395      0.5426
```

Select the ridge checks and check your gradient:

```
set_assignment(paste("https://github.com/MansMeg/IntroML/",
                     "blob/master/assignments/tests/assignment1/",
                     "assignment1-vg-ridge.yml", sep = ""))
mark_my_assignment(tasks = "nllr_grad")
```

3c. (1p) Implement the penalized loss NLL_r as `nllr(y, X, theta, lambda)`. Include the function code and your actual outputs for the three calls below; compare them with the supplied results to four decimal places.

```
nllr(y, X, theta = c(0,0,0), lambda = 0)

## [1] 0.6931
```

```
nllr(y, X, theta = c(0,0,0), lambda = 1)

## [1] 0.6931
```

```
nllr(y, X, theta = c(-1,0.5,0.5), lambda = 1)

## [1] 0.8613
```

With the ridge checks selected: `mark_my_assignment(tasks = "nllr")`

3d. (1p) Use `glmnet` to estimate `theta` for logistic ridge regression. Set `family = "binomial"`, `lambda = 1`, `alpha = 0`, and `standardize = FALSE` so the fit uses the same loss as your implementation on the already standardized predictors. Remove the intercept column from `X` when calling `glmnet`, and retain its default `intercept = TRUE` so it estimates an unpenalized intercept.

3e. (2p) Implement `gradient_descent_reg()` by adapting your `gradient_descent()` function. Reuse its batching and update logic, but use `nllr_grad()` for the gradient and `nllr()` for the loss, passing `lambda` to both. Start from $\theta = 0$. Return an `epochs` by `ncol(X)`$+2$ matrix with columns `epoch`, `nllr`, and the parameter names from `X`. Each row records the penalized loss on all training observations and the parameters after one complete epoch.

Complete this scaffold using your existing optimisation loop:

```
gradient_descent_reg <- function(y, X, batch_size, eta, epochs, lambda){
  results <- matrix(0.0, nrow = epochs, ncol = ncol(X) + 2L)
  colnames(results) <- c("epoch", "nllr", colnames(X))

  # Reuse your batching loop and subtract the nllr_grad() gradient.
  # Fill each row with the epoch, full-data nllr(), and parameters.

  return(results)
}
```

Run this function with batch sizes n and 10 for full GD and mini-batch GD, respectively. Reuse these runs in the next task, and include a short extract of the returned matrix with the input settings in your report.

Check: `mark_my_assignment(tasks = "gradient_descent_reg")`

3f. (2p) Try different learning parameters η and λ. When does the algorithm converge or diverge? Visualize the epochs (x-axis) and the penalized loss NLL_r evaluated on all observations (y-axis). Show at least one plot per algorithm that converges.

4 * Implementation of the Adam optimizer

For the optional distinction task, complete either this section or Section 3.

Reuse the binary-admission data from Section 1.

```
library(uuml)
data("binary")

binary$gre_sd <- (binary$gre - mean(binary$gre))/sd(binary$gre)
binary$gpa_sd <- (binary$gpa - mean(binary$gpa))/sd(binary$gpa)
X <- model.matrix(admit ~ gre_sd + gpa_sd, binary)
y <- binary$admit
```

4a. (2p) Implement the Adam mini-batch gradient descent algorithm (see Ruder, 2016, for details and psuedo-code) and (Kingma and Ba, 2014, for details) as an R function. Not that in Kingma and Ba (2014), the stepsize is denoted by α.

Note! Each epoch should include all data points. A tip is to order the data points randomly in each iteration.

Use the numerical constant $\epsilon = 10^{-8}$ in the denominator. Retain the first and second moments across epochs, and advance the bias-correction step once per mini-batch update.

Example: Which counter goes in the bias correction? *No submission required*

Adam corrects for zero initialization of the moments using

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}.$$

Here t counts mini-batch updates of the whole parameter vector. With 400 observations and batch size 10, epoch 1 uses $t = 1, \ldots, 40$; the first update of epoch 2 uses $t = 41$, not 2. Carry the counter and moments across epochs; initialize them to zero only for a new run.

A boilerplate for the algorithm can be found here.

```
adam <- function(y, X, sample_size, eta, beta1, beta2, epochs){
  # Setup output matrix
  results <- matrix(0.0, ncol = ncol(X) + 2L, nrow = epochs)
  colnames(results) <- c("epoch", "nll", colnames(X))

  # Run algorithm
  theta <- rep(0.0, ncol(X)) # Init theta to the 0 vector
  for(j in 1:epochs){

    ### Put the algorithm code here ###

    # Store epoch, mean NLL and parameter values
    results[j, "epoch"] <- j
    results[j, "nll"] <- nll(y, X, theta)
    results[j, -(1:2)] <- theta
  }
  return(results)
}
```

Select the Adam checks and check your function:

```
set_assignment(paste("https://github.com/MansMeg/IntroML/",
                     "blob/master/assignments/tests/assignment1/",
                     "assignment1-vg-adam.yml", sep = ""))
mark_my_assignment(tasks = "adam")
```

For example, run the baseline as follows:

```
set.seed(4711)
baseline <- adam(y, X, sample_size = 10, eta = 0.01,
                 beta1 = 0.9, beta2 = 0.999, epochs = 500)
```

4b. Print the function once in your report.

4c. Run these three settings using fresh calls with batch size 10 and 500 epochs. Set the seed to 4711 before each call.

Setting	η	β_1	β_2
Baseline	0.01	0.9	0.999
Larger learning rate	0.1	0.9	0.999
Longer gradient memory	0.01	0.99	0.999

Submit *one figure with two panels*, overlaying the three runs and adding the corresponding `glm()` reference lines. Reuse the plotting hint above to show:

4c.i. (1p) Full-training-data mean NLL against epoch.

4c.ii. (1p) One coefficient against epoch.

Briefly compare each modified run with the baseline: how do the learning rate and gradient memory affect progress and fluctuations? Small observed differences are valid.

References

Sebastian Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.

Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media, 2009.

Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.